3 arquiteturas RAG que todo desenvolvedor de prática conhecer: um guia completo com exemplos.

Traduzido para o português (Brasil) e chegando em seguida: pt-br

Fine-tuning
Prompt Engineering
RAG

A maioria das RAG em nível de produção não se baseia em uma única arquitetura. Elas combinam três padrões distintos, cada um resolvendo um problema diferente. Aqui está o que cada um faz e quando você precisa de cada um.

Arquitetura que não altera do longuinho para construir estatísticas de IA de produção.

Traduzido para o português (Brasil) e chegando em seguida: pt-br

O que é RAG e por que isso é realmente importante?

Perguntas Básicas no Aquecimento de Entrevista

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Antes de ser um desenvolvedor de aplicativos, vamos revisitar o que isso significa.

Os modelos de linguagem grande (LLMs) são treinados em dados da internet pública. Eles são ótimos em raciocínio, escrita e tarefas gerais. Eles não conhecem seus documentos internos, seus dados de produção ou qualquer coisa que tenha ocorrido após seu ponto de treinamento. A RAG está inferindo o que o modelo precisa de contexto específico em tempo de consulta e fornecê-lo antes que o modelo responda a uma pergunta.

Existem três maneiras de fazer isso, e elas se baseiam umas nas outras:

- Prompt Engineering: você escreve instruções melhores e mais específicas.
- Fine-tuning: você treina o modelo em seus dados para que ele aprenda seus padrões.
- RAG: você busca informações relevantes em tempo de consulta e as injeta no prompt.

O que é RAG e por que isso é realmente importante?

Perguntas Básicas no Aquecimento de Entrevista

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Vamos dar um exemplo.

Um hospital quer que um LLM responda perguntas sobre protocolos de tratamento. Os três cenários abaixo mostram por que cada abordagem é a ferramenta certa para o trabalho.

- Prompt Engineering: o hospital quer que o modelo responda no estilo de um protocolo específico. Eles escrevem uma instrução que especifica o formato, o tom e as regras. Rápido, barato, sem treinamento.
- Fine-tuning: o hospital quer que o modelo entenda a terminologia específica de seus pacientes e seus padrões de documentação. Eles treinam o modelo em milhares de exemplos para ensinar-lhe o estilo e o raciocínio específicos.
- RAG: o hospital quer que o modelo responda com base em seus protocolos atuais, que mudam toda semana. Em vez de treinar o modelo em cada versão, eles recuperam o protocolo relevante em tempo de consulta e o injetam no prompt.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Contexto:

- Prompt Engineering: você escreve instruções específicas e específicas em linguagem natural. Não é necessário treinamento ou infraestrutura. Rápido de implantar, barato de iterar, limitado pelo que o modelo já sabe e pelo quanto contexto você pode fornecer.
- Fine-tuning: você treina o modelo em seus dados para que ele aprenda padrões, terminologia e raciocínio específicos. Custo inicial de treinamento, latência de inferência mais baixa, não é necessário fornecer instruções em cada prompt.
- RAG: você busca informações relevantes em tempo de consulta e as injeta no prompt. O modelo não precisa saber nada sobre o domínio — ele apenas precisa ler o que você lhe fornecer.

Contexto:

- Fine-tuning: custo inicial de treinamento, latência de inferência mais baixa, não é necessário fornecer instruções em cada prompt.
- RAG: custo de recuperação em cada consulta, latência adicional em cada solicitação, precisa de uma infraestrutura de pesquisa em execução.

Exemplo:

- Prompt Engineering: "Responda como um protocolo médico. Use apenas os nomes dos medicamentos genéricos. Inclua a dosagem em mg."
- Fine-tuning: treinar em 10.000 exemplos de notas de alta hospitalar para que o modelo aprenda o formato, o tom e o raciocínio do hospital para um caso específico.
- RAG: buscar o protocolo de tratamento atual para o paciente específico e injetá-lo no prompt antes que o modelo responda.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Quando cada um falha:

- Prompt Engineering falha quando o modelo não tem o conhecimento de domínio de que precisa ou quando o formato que você descreve é muito complexo para ser descrito em um prompt.
- Fine-tuning falha quando os dados de treinamento estão desatualizados, são escassos ou não representam o caso de borda que você está tentando corrigir.
- RAG falha quando a etapa de recuperação não consegue encontrar o documento correto, quando os documentos recuperados são barulhentos ou quando o modelo ignora o contexto que você lhe fornece.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

O que é RAG e por que isso é realmente importante?

Perguntas Básicas no Aquecimento de Entrevista

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Vamos dar um exemplo.

Um hospital quer que um LLM responda perguntas sobre protocolos de tratamento. Os três cenários abaixo mostram por que cada abordagem é a ferramenta certa para o trabalho.

- Prompt Engineering: o hospital quer que o modelo responda no estilo de um protocolo específico. Eles escrevem uma instrução que especifica o formato, o tom e as regras. Rápido, barato, sem treinamento.
- Fine-tuning: o hospital quer que o modelo entenda a terminologia específica de seus pacientes e seus padrões de documentação. Eles treinam o modelo em milhares de exemplos para ensinar-lhe o estilo e o raciocínio específicos do hospital para um caso específico.
- RAG: o hospital quer que o modelo responda com base em seus protocolos atuais, que mudam toda semana. Em vez de treinar o modelo em cada versão, eles recuperam o protocolo relevante em tempo de consulta e o injetam no prompt antes que o modelo responda.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Contexto:

- Prompt Engineering: custo de tokens em cada solicitação, sem custo de treinamento, latência mínima, não é necessário fornecer instruções em cada prompt.
- Fine-tuning: custo inicial de treinamento, latência de inferência mais baixa, não é necessário fornecer instruções em cada prompt.
- RAG: custo de recuperação em cada consulta, latência adicional em cada solicitação, precisa de uma infraestrutura de pesquisa em execução.

Exemplo:

- Prompt Engineering: "Responda como um protocolo médico. Use apenas os nomes dos medicamentos genéricos. Inclua a dosagem em mg."
- Fine-tuning: treinar em 10.000 exemplos de notas de alta hospitalar para que o modelo aprenda o formato, o tom e o raciocínio do hospital para um caso específico.
- RAG: buscar o protocolo de tratamento atual para o paciente específico e injetá-lo no prompt antes que o modelo responda.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Quando cada um falha:

- Prompt Engineering falha quando o modelo não tem o conhecimento de domínio de que precisa ou quando o formato que você descreve é muito complexo para ser descrito em um prompt.
- Fine-tuning falha quando os dados de treinamento estão desatualizados, são escassos ou não representam o caso de borda que você está tentando corrigir.
- RAG falha quando a etapa de recuperação não consegue encontrar o documento correto, quando os documentos recuperados são barulhentos ou quando o modelo ignora o contexto que você lhe fornece.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

3 arquiteturas RAG que todo desenvolvedor de prática conhecer: um guia completo com exemplos.

Traduzido para o português (Brasil) e chegando em seguida: pt-br

Arquitetura que não altera do longuinho para construir estatísticas de IA de produção.

Traduzido para o português (Brasil) e chegando em seguida: pt-br

O que é RAG e por que isso é realmente importante?

Perguntas Básicas no Aquecimento de Entrevista

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Antes de ser um desenvolvedor de aplicativos, vamos revisitar o que isso significa.

Os modelos de linguagem grande (LLMs) são treinados em dados da internet pública. Eles são ótimos em raciocínio, escrita e tarefas gerais. Eles não conhecem seus documentos internos, seus dados de produção ou qualquer coisa que tenha ocorrido após seu ponto de treinamento. A RAG está inferindo o que o modelo precisa de contexto específico em tempo de consulta e fornecê-lo antes que o modelo responda a uma pergunta.

Existem três maneiras de fazer isso, e elas se baseiam umas nas outras:

- Prompt Engineering: você escreve instruções melhores e mais específicas.
- Fine-tuning: você treina o modelo em seus dados para que ele aprenda seus padrões.
- RAG: você busca informações relevantes em tempo de consulta e as injeta no prompt.

O que é RAG e por que isso é realmente importante?

Perguntas Básicas no Aquecimento de Entrevista

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Vamos dar um exemplo.

Um hospital quer que um LLM responda perguntas sobre protocolos de tratamento. Os três cenários abaixo mostram por que cada abordagem é a ferramenta certa para o trabalho.

- Prompt Engineering: o hospital quer que o modelo responda no estilo de um protocolo específico. Eles escrevem uma instrução que especifica o formato, o tom e as regras. Rápido, barato, sem treinamento.
- Fine-tuning: o hospital quer que o modelo entenda a terminologia específica de seus pacientes e seus padrões de documentação. Eles treinam o modelo em milhares de exemplos para ensinar-lhe o estilo e o raciocínio específicos do hospital para um caso específico.
- RAG: o hospital quer que o modelo responda com base em seus protocolos atuais, que mudam toda semana. Em vez de treinar o modelo em cada versão, eles recuperam o protocolo relevante em tempo de consulta e o injetam no prompt antes que o modelo responda.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Contexto:

- Prompt Engineering: custo de tokens em cada solicitação, sem custo de treinamento, latência mínima, não é necessário fornecer instruções em cada prompt.
- Fine-tuning: custo inicial de treinamento, latência de inferência mais baixa, não é necessário fornecer instruções em cada prompt.
- RAG: custo de recuperação em cada consulta, latência adicional em cada solicitação, precisa de uma infraestrutura de pesquisa em execução.

Exemplo:

- Prompt Engineering: "Responda como um protocolo médico. Use apenas os nomes dos medicamentos genéricos. Inclua a dosagem em mg."
- Fine-tuning: treinar em 10.000 exemplos de notas de alta hospitalar para que o modelo aprenda o formato, o tom e o raciocínio do hospital para um caso específico.
- RAG: buscar o protocolo de tratamento atual para o paciente específico e injetá-lo no prompt antes que o modelo responda.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Quando cada um falha:

- Prompt Engineering falha quando o modelo não tem o conhecimento de domínio de que precisa ou quando o formato que você descreve é muito complexo para ser descrito em um prompt.
- Fine-tuning falha quando os dados de treinamento estão desatualizados, são escassos ou não representam o caso de borda que você está tentando corrigir.
- RAG falha quando a etapa de recuperação não consegue encontrar o documento correto, quando os documentos recuperados são barulhentos ou quando o modelo ignora o contexto que você lhe fornece.

Fine-tuning ensina o modelo como pensar. RAG o fornece com os fatos de que ele precisa no momento em que precisa deles.

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

1. RAG básico: Como funciona

Sistema de Perguntas e Respostas

Fonte: https://blog.langchain.com/langgraph-the-llm-agent-runtime/

Sistema de Perguntas e Respostas com Recuperação Aumentada por Geração (RAG) em produção. Os dois quadros tracejados à direita são opcionais e são ativados quando a qualidade da recuperação falha.

Componentes:

- Recuperador: busca vetorial para encontrar trechos relevantes.
- Reranqueador (opcional): reordena os trechos recuperados para colocar os mais relevantes no topo.
- Gerador: LLM que lê os trechos e gera a resposta.
- Avaliador de relevância (opcional): verifica se os trechos recuperados são realmente úteis para a pergunta.
- Verificador de alucinação (opcional): verifica se a resposta é fundamentada nos trechos recuperados.

Fluxo:

- Pergunta → Recuperador → Reranqueador → Gerador → Resposta
- Se o avaliador de relevância sinalizar baixa relevância: reescrever a pergunta e tentar novamente.
- Se o verificador de alucinação sinalizar não fundamentado: gerar novamente ou escalar.

Quando falha:

- Recuperador retorna trechos irrelevantes → o gerador alucina a partir do ruído.
- Pergunta ambígua → recuperador busca o tópico errado.
- Documento necessário não está no corpus → nenhuma quantidade de melhoria ajuda.

Custo: Custo de recuperação em cada consulta, latência adicional em cada solicitação, precisa de uma infraestrutura de pesquisa em execução.

2. Fluxo de trabalho de pesquisa: Como funciona

Pesquisador Autônomo

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Pesquisador autônomo que decompõe uma pergunta em subtópicos, busca em cada um, avalia os resultados e sintetiza um relatório com citações.

Componentes:

- Planejador: decompõe a pergunta em subtópicos.
- Pesquisador: executa uma busca em cada subtópico.
- Avaliador: pontua a qualidade dos resultados da pesquisa.
- Sintetizador: escreve o relatório final com citações.

Fluxo:

- Pergunta → Planejador → [Pesquisador × N] → Avaliador → Sintetizador → Relatório
- Se a cobertura estiver baixa: gerar consultas de acompanhamento e repetir.

Quando falha:

- Planejador perde um subtópico crítico → ponto cego no relatório.
- Pesquisadores recuperam fontes de baixa qualidade → síntese fundamentada em lixo.
- Avaliador aceita cobertura ruim → usuário recebe um relatório incompleto.

Custo: Muito alto. Cada pesquisa aciona dezenas de chamadas de LLM e buscas. Não é adequado para consultas em tempo real.

3. Agente Reconhecedor de Falhas: Como funciona

Agente Reconhecedor de Falhas

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Agente Reconhecedor de Falhas com um ciclo de feedback. O crítico avalia a recuperação e aciona novas tentativas quando a qualidade falha.

Componentes:

- Planejador: decompõe a consulta em subtópicos.
- Pool de agentes de recuperação: busca em paralelo em vários índices.
- Crítico: avalia a qualidade da recuperação e decide: aprovar, repetir ou escalar.
- Gerador: sintetiza a resposta a partir dos documentos aprovados.

Fluxo:

- Consulta → Planejador → [Agentes de recuperação] → Crítico → Gerador → Resposta
- Se o crítico reprovar: reformular a consulta e tentar novamente.
- Se o crítico for reprovado novamente: escalar ou retornar o que for melhor.

Quando falha:

- Crítico aceita recuperação ruim → alucinação a partir de fontes ruins.
- Crítico rejeita tudo → loop infinito consumindo latência e custo.
- Planejador cria subtópicos ruins → agentes recuperam o que não é necessário.

Custo: Muito alto. Múltiplas chamadas de LLM para planejamento, avaliação e síntese, mais todas as buscas. Não é adequado para latência em tempo real.

4. Graphrag: Como funciona

Graphrag: Recuperação Aumentada por Grafo

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Graphrag: Recuperação Aumentada por Grafo com detecção de comunidade e resumos hierárquicos.

Componentes:

- Extrator de grafo: extrai entidades e relacionamentos do corpus.
- Detector de comunidade: agrupa nós em clusters temáticos.
- Gerador de resumo: pré-computa resumos de comunidade em vários níveis.
- Recuperador: busca em resumos de comunidade e subgrafos.
- Gerador: LLM que lê o contexto recuperado e gera a resposta.

Fluxo:

- Corpus → Extrator de grafo → Detector de comunidade → Gerador de resumo → Grafo + resumos
- Consulta → Recuperador → Subgrafo + resumos → Gerador → Resposta

Quando falha:

- Consulta não mapeia para nenhuma entidade → recuperação retorna ruído.
- Corpus não tem estrutura de entidade → o grafo é barulhento e esparso.
- Consulta exige conteúdo literal → resumos perdem o texto exato.

Custo: Muito alto. Pré-processamento pesado para extração de grafo e resumos, mais custo de consulta em cada solicitação.

Como escolher na prática (6 estruturas de decisão)

Fonte: https://www.linkedin.com/posts/vishakha-sadhwani_rag-vs-fine-tuning-vs-prompt-engineering-activity-7391...

Comece com o RAG básico.

Se o seu caso de uso for bem coberto pelo conhecimento de treinamento do modelo, adicione um reranqueador e melhore a qualidade da recuperação antes de adicionar qualquer coisa mais complexa.

Adicione um avaliador de relevância quando:

- Consultas ambíguas são comuns.
- Seus corpos são heterogêneos.
- Você precisa de recuperação de qualidade.

Adicione um verificador de alucinação quando:

- Precisão factual é inegociável.
- Você está implantando em domínios médicos, jurídicos ou financeiros.
- Você precisa de citações em nível de sentença.

Mude para um fluxo de trabalho quando:

- Perguntas exigem síntese de múltiplas fontes.
- Você precisa de cobertura de tópicos, não apenas de relevância.
- Latência não é uma restrição.

Mude para o Graphrag quando:

- Perguntas dependem de relacionamentos entre entidades.
- Você precisa de resumos em nível de corpus.
- Custo de pré-processamento é aceitável.

Regra prática: Comece com o RAG básico. Adicione complexidade somente quando os modos de falha específicos exigirem isso